Most Influential Multi-type Community Search over Heterogeneous Information Networks

Technical Report

Jun Zhao
Swinburne University of Technology
Melbourne, Australia
junzhao@swin.edu.au

Lu Chen
Swinburne University of Technology
Melbourne, Australia
luchen@swin.edu.au

Chengfei Liu*
Swinburne University of Technology
Melbourne, Australia
cliu@swin.edu.au

Rui Zhou
Swinburne University of Technology
Melbourne, Australia
rzhou@swin.edu.au

ABSTRACT

Community search over heterogeneous information networks (HINs) finds a cohesive community containing a given query vertex. Recent studies further consider the influence of a community, but they typically return communities of a single vertex type and measure influence by aggregating predefined importance values of individual vertices. In real HINs, however, influence is propagated collectively by community members of different types, and the contribution of each member depends on the others because their propagation ranges overlap. In this paper, we study Most Influential Multi-type Community Search (MIMCS) over HINs: given a query vertex, a set of relational constraints, and a budget on vertex costs, MIMCS finds a connected multi-type community containing the query that maximizes the collective propagation influence over a target vertex type. We propose a propagation model based on multiple meta-paths, in which members of different types in the community jointly seed the diffusion on the target type, show that the resulting influence function is monotone and submodular, and prove that MIMCS is NP-hard. To solve it exactly, we develop MIMC-B&B, a branch-and-bound algorithm equipped with completion-cost reduction, completion-aware influence bounding, and deficit-guided branching: we derive a lower bound on the cost that any feasible completion of a partial community must still pay to satisfy its unresolved relational constraints, including those induced by the supporting vertices themselves, and use it to prune states that cannot be completed within the budget; the same bound reserves the unavoidable completion cost in an upper bound on the attainable influence, which prunes states that cannot improve the current best solution; and the search branches first on the most constrained unresolved requirement. Extensive experiments on five real-world HINs demonstrate the efficiency of MIMC-B&B and the effectiveness of our community model.

PVLDB Reference Format:
Jun Zhao, Lu Chen, Chengfei Liu, and Rui Zhou. Most Influential Multi-type Community Search over Heterogeneous Information Networks. PVLDB, 20(1): XXX-XXX, 2027.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/junzhaoswin/MIMCS.

1 INTRODUCTION

In many real applications, the data are inherently heterogeneous. For instance, in bibliographic network data, authors write papers, papers are published in venues, and papers mention topics; in an e-commerce network, users purchase items sold by stores and organized into categories. Such network data are naturally modeled as heterogeneous information networks (HINs), in which vertices and edges are typed, and the semantics of a relationship between two vertices is described by a meta-path [30, 32].

Community search over HINs has drawn extensive attention from both academia and industry. Given a query vertex, it finds a cohesive community containing the query, where cohesiveness is captured by models such as $(k, \mathcal{P})$-core and $(k, \mathcal{P})$-truss [9, 10, 16], or by relational constraints that allow the community to span multiple vertex types [3, 15]. Apart from cohesiveness, one essential aspect of a community in an HIN is its influence, i.e., its ability to affect other entities of a given target type. For example, a research community is influential if its authors, papers, and venues can jointly influence many other authors.

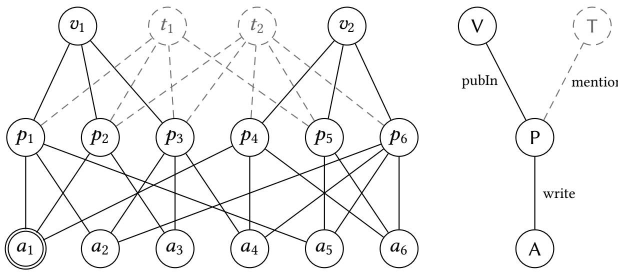
However, the influence of a community in an HIN has not been well captured. The existing influential community models over HINs measure a community by the importance values of its individual members, which may be derived through meta-paths, but the returned community consists of vertices of a single type [26, 47]. Propagation-based influence, which reflects how members jointly reach other vertices, has been studied in homogeneous social networks [22, 44], but not in HINs, where members of a community and the entities they reach can be of different types. Consider the following application scenario.

In a bibliographic network, a researcher may wish to form a research community to disseminate a research topic to other authors, i.e., the target type is author. Such a community consists of not only authors but also the papers and venues that support them, and its members are tied together: every author in the community writes some papers in the community, every paper is written by some authors and published in a venue in the community, and every venue publishes several papers in the community.

Consider how the topic spreads from the community to other authors. Members of different types reach authors via instances of different meta-paths. An author reaches their co-authors, i.e., along meta-path (author, paper, author); a paper reaches its authors, i.e., along meta-path (paper, author); and a venue reaches

*Chengfei Liu is the corresponding author.

Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX



(a) An HIN (b) Schema

Figure 1: An example HIN and its schema

the authors of the papers it publishes, i.e., along meta-path (venue, paper, author). The authors reached in these ways are the entry points of the topic among the authors, and from them the topic further propagates along co-authorship. In other words, the members of the community jointly initiate a single propagation over the authors through multiple meta-paths, and the influence of the community is the number of authors eventually reached. Such influence is a collective outcome that cannot be inferred from the importance of individual members. Two authors who share most of their collaborators contribute little more together than either alone. A venue, although not an author, can bring in authors that no author member reaches. Moreover, engaging an entity incurs a cost, which differs across entities, e.g., involving a venue costs differently from inviting a co-author, and the community must be formed within a budget.

The scenario calls for a community that satisfies the cross-type structural requirements and maximizes the influence collectively propagated by its members through multiple meta-paths within a given budget.

Based on the needs in the scenario, we propose the problem of Most Influential Multi-type Community Search (MIMCS) over HINs, which is built on the following three components.

Multi-type cohesiveness. The community should span the types that support one another, as the authors, papers, and venues in the scenario do. We adopt the relational community model [15], in which a set of relational constraints specifies, for each pair of types of interest, how many neighbors of one type each member of the other type must have within the community, and the community is required to be connected. This lets users prescribe the cross-type structure through which the members support one another, and it also determines which non-target entities can join the community and hence take part in its propagation.

Influence modeling. A key issue is how to model the propagation influence of such a multi-type community. The existing propagation models, such as the independent cascade and linear threshold models [4, 5, 17], and the reverse-reachable (RR) set techniques that estimate the expected influence [2, 36, 37], are designed for homogeneous graphs. There, the seeds and the vertices to be activated are of the same type and connected by direct edges, and how a paper or a venue triggers propagation among authors is not defined. We bridge this gap by a propagation model based on multiple meta-paths. Following the meta-path semantics of HINs [32], the propagation among the target vertices is carried by a symmetric meta-path, e.g., (author, paper, author), along which we project the HIN onto a target graph and run the independent cascade model. The community members enter this propagation as entry seeds. A member of the target type is an entry seed itself, while a member of another type contributes the target vertices it reaches along a set of prescribed asymmetric meta-paths, e.g., (venue, paper, author). The entry seeds of all members jointly seed the diffusion on the target graph, and the influence of the community is estimated from them using RR sets. We show that the resulting influence function is monotone and submodular.

Budget and objective. Each vertex is associated with a cost, and the community is subject to a budget on its total cost. Putting the three components together, given a query vertex (or, more generally, a set of query vertices), a set of relational constraints, a target vertex type, vertex costs, and a budget, MIMCS finds a connected multi-type subgraph that contains the query, satisfies the relational constraints within the budget, and maximizes the collective propagation influence over the target type.

We prove that MIMCS is NP-hard by a reduction from the clique problem. The hardness already lies in feasibility: unless P = NP, no polynomial-time algorithm is guaranteed to find a feasible community, and consequently no polynomial-time approximation algorithm exists for any approximation ratio. This holds even though the influence function is monotone and submodular, and therefore the greedy approach commonly used for influence maximization does not apply; an exact algorithm is needed. Moreover, the search techniques of existing influential community search cannot be applied directly. In homogeneous social networks, the candidate communities, e.g., maximal kr-cliques [22], are finite, independent of the query, and can be computed offline and indexed, so the search only needs to bound the influence of each whole candidate community. In MIMCS, a candidate community is a subset of vertices selected around the query under relational constraints and a budget, and the number of such subsets grows exponentially with the size of the neighborhood of the query. Bounding must therefore be performed while a community is constructed, and this is nontrivial. A partial community that currently violates a relational constraint cannot be discarded, because adding suitable members may still make it feasible. These additional members consume the budget and may themselves introduce new constraints to be satisfied. Hence, pruning a partial community requires reasoning about whether it can still be completed and how much any completion must cost.

To address the challenges, we first develop Exhaustive Search, an exact baseline that systematically enumerates the connected communities containing the query. Then, we develop MIMC-B&B, a branch-and-bound algorithm on the same enumeration framework, built around the cost of completing a partial community. Its completion-cost reduction derives a lower bound on the additional cost that any feasible completion must incur to satisfy the unresolved relational constraints, and prunes a search state when this cost exceeds the remaining budget. Its completion-aware influence bounding reuses this completion cost to tighten the upper bound on the attainable influence, since the budget reserved for structural completion cannot be spent on additional influence. Its deficit-guided branching selects the most constrained unresolved requirement first, so that restrictive search decisions are exposed as early as possible. Throughout the search, the RR sets on the target graph are sampled once and shared by all search states, so that the influence and its bounds are evaluated by set coverage; both algorithms are exact with respect to this RR-set estimate of influence. The contributions of this work are summarized as follows.

- We propose the Most Influential Multi-type Community Search (MIMCS) problem over HINs, which finds a query-centered multi-type relational community that satisfies a vertex-cost budget and maximizes its collective propagation influence over a target vertex type.
- We propose a propagation-based influence model for multi-type communities over HINs, which extends the propagation models and RR-set estimation of homogeneous graphs by carrying the propagation along a symmetric meta-path and letting members of non-target types seed it along asymmetric meta-paths. We show that the influence function is monotone and submodular, and we prove that MIMCS is NP-hard and, unless P = NP, admits no polynomial-time approximation algorithm for any ratio.
- We develop an exact enumeration framework, Exhaustive Search, and on top of it MIMC-B&B, a branch-and-bound algorithm equipped with completion-cost reduction, completion-aware influence bounding, and deficit-guided branching, which prunes partial communities that cannot be completed within the budget or cannot improve the current best solution.
- We conduct extensive experiments on five real-world HINs. The results show that MIMC-B&B substantially reduces the running time of Exhaustive Search and solves more queries to optimality within the time limit, and that the three techniques provide complementary gains. A case study on DBLP shows that MIMCS returns the heterogeneous entities that jointly support the influence of a query-centered community.

The remainder of this paper is organized as follows. Section 2 formulates MIMCS. Section 3 presents the propagation-based influence model and establishes the hardness of MIMCS. Section 4 presents Exhaustive Search, and Section 5 develops MIMC-B&B. Section 6 reports the experimental results. Section 7 discusses related work, and Section 8 concludes the paper.

2 PROBLEM STATEMENT

In this section, we introduce the necessary background on HINs and relational communities, and then formulate the MIMCS problem. Table 1 summarizes the frequently used notation.

2.1 Preliminaries

Definition 1. **(HIN) [30]:** An HIN is an undirected graph $\mathcal{H} = (V, E)$ with a vertex-type mapping function $\psi : V \rightarrow \mathcal{A}$ and an edge-type mapping function $\phi : E \rightarrow \mathcal{R}$, where each vertex $v \in V$ belongs to a vertex type $\psi(v) \in \mathcal{A}$ and each edge $e \in E$ belongs to an edge type $\phi(e) \in \mathcal{R}$.

For any $V' \subseteq V$, let $\mathcal{H}[V']$ denote the subgraph of $\mathcal{H}$ induced by V', that is, the subgraph with vertex set V' and edge set $\{\{u, v\} \in E \mid u, v \in V'\}$. For a vertex v, let $\mathcal{N}_{\mathcal{H}}(v) = \{u \in V \mid \{u, v\} \in E\}$ denote its neighbors and $d(v) = |\mathcal{N}_{\mathcal{H}}(v)|$ its degree. For a vertex type A, let $\mathcal{N}_{\mathcal{H}}(v, A) = \{u \in \mathcal{N}_{\mathcal{H}}(v) \mid \psi(u) = A\}$ denote the type-A neighbors of v, and let $d(v, A) = |\mathcal{N}_{\mathcal{H}}(v, A)|$ be its type-A degree.

Definition 2. **(HIN Schema):** Given an HIN, $\mathcal{H} = (V, E)$, with mappings $\psi : V \rightarrow \mathcal{A}$ and $\phi : E \rightarrow \mathcal{R}$, its schema $T_{\mathcal{H}}$ is a directed graph defined over vertex types $\mathcal{A}$ and edge types (as relations) from $\mathcal{R}$, i.e., $T_{\mathcal{H}} = (\mathcal{A}, \mathcal{R})$.

The schema of an HIN specifies the allowable edge types between all vertex types. If the schema contains a relation R from vertex type A to type B, the inverse relation R^{-1} from B to A is also defined. We use lower-case letters (e.g., a_1) to denote vertices and upper-case letters (e.g., A) to denote vertex types.

Definition 3. **(Meta-path) [32]:** A meta-path $\mathcal{P}$ is a path defined on the HIN schema $T_{\mathcal{H}} = (\mathcal{A}, \mathcal{R})$, written as $\mathcal{P} = A_1 \xrightarrow{R_1} A_2 \xrightarrow{R_2} \cdots \xrightarrow{R_\ell} A_{\ell+1}$, where $A_i \in \mathcal{A}$ and $R_i \in \mathcal{R}$. We call A_1 and $A_{\ell+1}$ its starting and ending types, respectively.

When no multiple relation types exist between the same pair of vertex types, we denote $\mathcal{P}$ by its vertex-type sequence, i.e., $\mathcal{P} = (A_1, A_2, \ldots, A_{\ell+1})$. The left and right halves of $\mathcal{P}$ are denoted by $l(\mathcal{P}) = A_1 \xrightarrow{R_1} \cdots \xrightarrow{R_{\lceil(\ell-1)/2\rceil}} A_{\lceil(\ell+1)/2\rceil}$ and $r(\mathcal{P}) = A_{\ell+1} \xrightarrow{R_\ell^{-1}} \cdots \xrightarrow{R^{-1}_{\lceil(\ell+1)/2\rceil}} A_{\lceil(\ell+1)/2\rceil}$, respectively. We say $\mathcal{P}$ is symmetric if $l(\mathcal{P}) = r(\mathcal{P})$; otherwise it is asymmetric. Given a meta-path $\mathcal{P} = A_1 \xrightarrow{R_1} \cdots \xrightarrow{R_\ell} A_{\ell+1}$, a vertex sequence $I = (z_0, \ldots, z_\ell)$ is a *path instance* of $\mathcal{P}$ if $\psi(z_i) = A_{i+1}$ for $0 \leq i \leq \ell$, and each edge $\{z_{i-1}, z_i\} \in E$ follows relation R_i for $1 \leq i \leq \ell$.

Definition 4. **(Relational Constraint):** A relational constraint s is a triplet $\langle A_1, A_2, k\rangle$ with $A_1, A_2 \in \mathcal{A}$ and $k \geq 1$, requiring every vertex of type A_1 to have at least k type-A_2 neighbors. Given a graph $\mathcal{H}'$, a vertex v satisfies s in $\mathcal{H}'$ if (1) $\psi(v) \neq A_1$ or (2) $|\mathcal{N}_{\mathcal{H}'}(v, A_2)| \geq k$.

Relational constraint set. A set of relational constraints S jointly specifies the typed neighborhood requirements of a community. To avoid redundant constraints, for each ordered type pair (A_i, A_j), we retain only the constraint with the largest k if multiple constraints are specified for the same pair. We further require every constrained type pair to be represented in both directions: if $\langle A_i, A_j, k\rangle \in S$ but no constraint is specified for the reverse pair (A_j, A_i), we include $\langle A_j, A_i, 1\rangle$ in S. This requirement ensures that a vertex selected to support another type is itself connected to at least one selected vertex of that type.

Let $\mathcal{A}_S = \{A \in \mathcal{A} \mid \exists\langle A_i, A_j, k\rangle \in S,\ A = A_i \text{ or } A = A_j\}$ be the set of vertex types involved in S, and let $V_S = \{v \in V \mid \psi(v) \in \mathcal{A}_S\}$ be the set of vertices of these types.

Definition 5. **(Relational Community) [15]:** Given a relational constraint set S and a vertex set $Y \subseteq V_S$, the induced subgraph $\mathcal{H}[Y]$ is a *relational community* (r-com) under S if it is connected and every vertex $v \in Y$ satisfies all constraints in S within $\mathcal{H}[Y]$.

Since $\mathcal{H}[Y]$ is determined by Y, we refer to an r-com by its vertex set when no confusion arises.

We adopt the relational community model as the structural requirement of MIMCS. It lets users specify, for the vertex types of their choice, how many neighbors of another type each member must have, so that the returned community spans multiple types whose members support one another as prescribed.

Table 1: Notations

Notation	Meaning
$\mathcal{H} = (V, E)$, ψ, ϕ	an HIN and its vertex- and edge-type mappings
$\mathcal{P}$, $\mathcal{P}_s$, $\mathcal{P}_a$	a meta-path, the symmetric target meta-path, and prescribed asymmetric entry meta-paths
S, $\mathcal{A}_S$, V_S	the relational constraint set, the vertex types it involves, and their vertices
Q_0	the nonempty query set anchoring the community
$c(v)$, b	the cost of vertex v and the budget
A_T, T, G_T	the target type, the target vertices, and the projected target graph
$\Gamma_T(X)$	the target entry set induced by X
$\mathcal{J}$, $f(X)$	the RR-set collection and the influence score of X
$\mathrm{cov}(v)$	the set of RR sets covered by v
(Q, C)	a search state: the partial community Q and its residual domain C
$\partial_C(Q)$	the boundary of Q in C
$\delta_Q(u, A)$, $P_C(u, A)$	the deficit of u for type A in Q and its supporter pool in C
$\beta(Q)$	the residual budget $b - c(Q)$
$LB_{\mathrm{comp}}(Q, C)$	the completion-cost lower bound of (Q, C)
$UB_{\mathrm{CA}}(Q, C)$, $UB_{\mathrm{SC}}(Q, C)$	the completion-aware and support-constrained influence upper bounds of (Q, C); $UB = \min\{UB_{\mathrm{CA}}, UB_{\mathrm{SC}}\}$

2.2 Problem Definition

Relational communities capture cross-type cohesion, but structural feasibility alone determines neither how influential a community is nor how costly it is to engage its members. We therefore evaluate a community by its collective propagation influence, in which the contribution of a member depends on the other selected members through their overlapping propagation reach, and we bound the total cost of its members by a budget.

Influence measurement. We use an integer-valued function $f : 2^V \to \mathbb{Z}_{\geq 0}$ as the influence score optimized by MIMCS. Conceptually, $f(X)$ estimates the expected propagation influence that a vertex set X collectively generates among the vertices of a target type A_T, where propagation between target vertices follows a prescribed symmetric meta-path $\mathcal{P}_s$ and non-target members reach target vertices through a prescribed set $\mathcal{P}_a$ of asymmetric meta-paths. Section 3 instantiates f on an HIN using this propagation model and standard RR sampling, shows that f is monotone and submodular, and proves that MIMCS is NP-hard.

Budget and cost. Engaging different members of a community requires different amounts of effort, which existing influential community search over HINs does not account for. We therefore associate each vertex v with a positive cost $c(v)$ and impose a budget b on the total cost of the community. For a vertex set Y, let $c(Y) = \sum_{v \in Y} c(v)$.

We are now ready to present the community model of MIMCS as follows.

Definition 6. **(Budgeted Multi-type Community):** Given an HIN $\mathcal{H} = (V, E)$, a relational constraint set S, a positive vertex-cost function c, and a budget $b > 0$, the induced subgraph $\mathcal{H}[Y]$ of a vertex set $Y \subseteq V_S$ is a *budgeted multi-type community (BMC)* if it satisfies both of the following conditions: (i) **Cohesiveness:** $\mathcal{H}[Y]$ is an r-com under S; and (ii) **Budget:** $c(Y) \leq b$.

Problem 1 (MIMCS Problem). Given an HIN $\mathcal{H}$, a relational constraint set S, a target vertex type A_T with prescribed meta-paths $\mathcal{P}_s$ and $\mathcal{P}_a$, a nonempty query set $Q_0 \subseteq V_S$, vertex costs c, a budget b, and the resulting influence function f, the *Most Influential Multi-type Community Search (MIMCS)* problem is to find a BMC $Y^\star$ containing Q_0 such that $f(Y^\star) \geq f(Y)$ for every BMC $Y \supseteq Q_0$.

We focus on the single-query setting $Q_0 = \{q\}$; supporting several query vertices requires a different connectivity maintenance during enumeration and is left for future work.

3 INFLUENCE MODEL FOR HIN

In this section, we first present the influence model for MIMCS, then establish its monotonicity and submodularity, and finally show that MIMCS is NP-hard and admits no polynomial-time approximation unless P = NP. The model evaluates communities on an RR-set collection that is sampled once and fixed thereafter; the exactness guarantees of our algorithms refer to the optimization of this fixed estimate, whose accuracy with respect to the expected influence is governed by the sample size.

3.1 Meta-Path Projection and Influence Model

Let A_T be the target vertex type of the query, and let $T = \{v \in V \mid \psi(v) = A_T\}$ be the set of target vertices, with $n_T = |T|$. Let $\mathcal{P}_s$ be the prescribed symmetric meta-path whose starting and ending types are both A_T. We project $\mathcal{H}$ onto the target vertices through $\mathcal{P}_s$, obtaining the projected target graph $G_T = (T, E_T)$, an undirected graph in which $\{u, v\} \in E_T$ for distinct $u, v \in T$ if and only if at least one instance of $\mathcal{P}_s$ connects u and v in $\mathcal{H}$. Only the existence of such instances is used to construct E_T; propagation probabilities are defined directly on G_T.

Although G_T is undirected, propagation on it is directional. We adopt the weighted-cascade (WC) parameterization of the independent cascade model [17] on G_T: an activation attempt from u to v succeeds with probability $p_T(u, v) = 1/d_{G_T}(v)$, where $d_{G_T}(v)$ is the degree of v in G_T. Consequently, the two directions of the same projected edge may have different activation probabilities.

To incorporate non-target community members, let $\mathcal{P}_a$ denote the prescribed set of asymmetric meta-paths whose ending type is A_T. For a non-target vertex v, its target entry set is $\Gamma_T(v) = \{u \in T \mid \text{some meta-path in } \mathcal{P}_a \text{ has an instance from } v \text{ to } u\}$; for a target vertex $v \in T$, let $\Gamma_T(v) = \{v\}$; and for any $X \subseteq V$, let $\Gamma_T(X) = \bigcup_{v \in X} \Gamma_T(v)$. Thus, non-target members contribute influence by injecting their target entries as seeds. This entry mapping is one-shot: once the entry seeds are determined, diffusion occurs only on G_T and never propagates back through non-target types. The mapping is deterministic, so a high-reach non-target member injects all of its target entries, and whether such members are affordable is left to the cost function. Assigning each entry an activation probability instead, and sampling the entry edges in the reverse traversal, would change only the sampling of $\mathcal{J}$ and leave f a coverage function, so the results of Sections 3–5 apply verbatim. Under the WC model on G_T, the target entry set $\Gamma_T(X)$ serves as the seed set of the propagation initiated by X. To estimate its expected propagation influence, we follow the standard reverse-reachable (RR) sampling framework [2, 36, 37]: an RR set is obtained by sampling a target vertex uniformly at random and collecting the vertices that reach it in a random realization of the WC model on G_T. Let $\mathcal{J}$ be an RR-set collection sampled on G_T and kept unchanged during the search. We define the influence score of X as $f(X) = \left|\{J \in \mathcal{J} \mid \Gamma_T(X) \cap J \neq \emptyset\}\right|$, the number of RR sets hit by the entry seeds of X. Since the standard RR estimator rescales $f(X)$ only by the constant factor $n_T/|\mathcal{J}|$, maximizing f is equivalent to maximizing the sampled estimate of the expected influence. The sample-size bounds of the RR framework [2, 36, 37] are stated for seed sets of a fixed size, whereas MIMCS would require the estimate to hold uniformly over exponentially many candidate BMCs; we therefore claim no uniform guarantee and treat the accuracy of the estimate as orthogonal to the exact optimization studied here.

Example 1. Consider the HIN in Figure 1 with $S = \{\langle A, P, 2\rangle, \langle P, A, 2\rangle, \langle P, V, 1\rangle, \langle V, P, 3\rangle\}$, query $Q_0 = \{a_1\}$, target type $A_T = A$, and budget $b = 20$. We use the symmetric meta-path $\mathcal{P}_s = (A, P, A)$, the asymmetric entry meta-paths $\mathcal{P}_a = \{(P, A), (V, P, A)\}$, and the RR-set collection $\mathcal{J} = \{\{a_1, a_2\}, \{a_2, a_5\}, \{a_1, a_5\}, \{a_3\}, \{a_4\}, \{a_6\}, \{a_3, a_4\}, \{a_4, a_6\}\}$, written $J_1, \ldots, J_8$. Each author a_i has $\Gamma_T(a_i) = \{a_i\}$ and covers the RR sets that contain a_i, with costs $c(a_1) = c(a_3) = 2$, $c(a_2) = 3$, and $c(a_4) = c(a_5) = c(a_6) = 4$; Table 2 lists the costs, target entry sets, and covered RR sets of the papers and venues. The community $Y^\star = \{a_1, a_2, a_3, p_1, p_2, p_3, v_1\}$ contains the query, satisfies all relational constraints, and has cost 20. Its entry seeds cover every RR set except J_6, giving $f(Y^\star) = 7$, and it is the optimal community under this budget.

Table 2: Costs, target entries, and covered RR sets

v	$c(v)$	$\Gamma_T(v)$	RR sets covered
p_1	3	a_1, a_2, a_5	J_1, J_2, J_3
p_2	3	a_1, a_3	J_1, J_3, J_4, J_7
p_3	4	a_2, a_3, a_4	$J_1, J_2, J_4, J_5, J_7, J_8$
p_4	4	a_1, a_4, a_6	$J_1, J_3, J_5, J_6, J_7, J_8$
p_5	5	a_5, a_6	J_2, J_3, J_6, J_8
p_6	6	a_2, a_4, a_5, a_6	all but J_4
v_1	3	a_1, a_2, a_3, a_4, a_5	all but J_6
v_2	7	a_1, a_2, a_4, a_5, a_6	all but J_4

Monotonicity and submodularity. For $v \in V$, let $\mathrm{cov}(v) = \{J \in \mathcal{J} \mid \Gamma_T(v) \cap J \neq \emptyset\}$ denote the set of RR sets covered by v. Since $\Gamma_T(X) = \bigcup_{v \in X} \Gamma_T(v)$, we have $\mathrm{cov}(X) = \bigcup_{v \in X} \mathrm{cov}(v)$ and $f(X) = |\mathrm{cov}(X)|$; that is, f is a coverage function over $\mathcal{J}$.

Proposition 1 (Monotonicity and Submodularity). *For any $X_1 \subseteq X_2 \subseteq V$ and any $u \in V \setminus X_2$, $f(X_1) \leq f(X_2)$ and $f(X_1 \cup \{u\}) - f(X_1) \geq f(X_2 \cup \{u\}) - f(X_2)$.*

Proof. Since $X_1 \subseteq X_2$ implies $\mathrm{cov}(X_1) \subseteq \mathrm{cov}(X_2)$, we have $f(X_1) \leq f(X_2)$, and the marginal gain $f(X \cup \{u\}) - f(X) = |\mathrm{cov}(u) \setminus \mathrm{cov}(X)|$ can only decrease as X grows. □

3.2 Problem Hardness

We now establish the hardness of MIMCS.

Theorem 1. *MIMCS is NP-hard.*

Proof. We reduce from Clique. Given an undirected graph $G = (V_G, E_G)$ with $|V_G| \geq 3$ and an integer $k \geq 2$, we construct an MIMCS instance as follows. The HIN has two vertex types A and B. For each vertex of G, we create a type-A vertex and preserve the edges of G among the type-A vertices; let V_A denote the set of type-A vertices. We add one type-B query vertex q adjacent to every vertex in V_A, and set $Q_0 = \{q\}$ and $S = \{\langle B, A, k\rangle, \langle A, B, 1\rangle, \langle A, A, k-1\rangle\}$. Every vertex has unit cost, and the budget is $b = k+1$. For the influence model, we take A as the target type, prescribe $\mathcal{P}_s = (A, B, A)$ and no asymmetric entry meta-path, and fix the RR collection $\mathcal{J} = \{\{v\} \mid v \in V_A\}$, which is a possible outcome of RR sampling on G_T because every target vertex has $|V_A| - 1 \geq 2$ projected neighbors and hence yields a singleton RR set with positive probability. Under this choice, $\Gamma_T(v) = \{v\}$ for every $v \in V_A$ and $\Gamma_T(q) = \emptyset$, so $f(Y) = |Y \cap V_A|$ for every vertex set Y. The construction is polynomial.

If G contains a k-clique K, then $K \cup \{q\}$ is a feasible community: it contains the query, is connected through q, satisfies $\langle B, A, k\rangle$ and $\langle A, B, 1\rangle$, and every vertex of K has the other $k-1$ vertices of K as type-A neighbors. Its cost is $k+1$.

Conversely, let Y be any feasible community of the constructed instance. Since $q \in Y$, the constraint $\langle B, A, k\rangle$ requires at least k type-A vertices in Y. Because all vertices have unit cost and $b = k+1$, Y contains exactly k type-A vertices. The constraint $\langle A, A, k-1\rangle$ then requires each of them to be adjacent to all the others, so they form a k-clique in G. Hence G contains a k-clique if and only if the constructed instance admits a feasible community, and therefore MIMCS is NP-hard. □

The same reduction encodes the hardness of MIMCS entirely in feasibility, which also rules out approximation algorithms.

Theorem 2. *Unless* P = NP*, no polynomial-time algorithm computes a feasible solution to MIMCS, that is, returns a BMC containing Q_0 whenever one exists. Consequently, for any $\alpha \in (0, 1]$, no polynomial-time algorithm returns, whenever a BMC containing Q_0 exists, a BMC Y with $f(Y) \geq \alpha f(Y^\star)$.*

Proof. Suppose that a polynomial-time algorithm $\mathcal{A}$ returns a BMC containing Q_0 whenever the given MIMCS instance admits one. We use $\mathcal{A}$ to decide Clique in polynomial time. Given an instance (G, k) of Clique, we construct the MIMCS instance in the proof of Theorem 1, run $\mathcal{A}$ on it, and check whether its output is a BMC containing q. The construction and the check take polynomial time, since verifying the connectivity, the relational constraints, and the budget of a vertex set is polynomial. If the output is such a BMC, then its type-A vertices form a k-clique of G, as shown in the proof of Theorem 1. Otherwise, the constructed instance admits no BMC containing q, because $\mathcal{A}$ would have returned one, and hence G contains no k-clique. Thus Clique would be decidable in polynomial time, which implies P = NP.

For the second claim, let $\alpha > 0$ and suppose that a polynomial-time α-approximation algorithm exists, that is, an algorithm that, whenever a feasible BMC exists, returns a BMC Y containing Q_0 with $f(Y) \geq \alpha f(Y^\star)$. Such an algorithm in particular returns a feasible BMC whenever one exists, which contradicts the first claim. Since the argument uses only feasibility, it holds for every value of α and already applies to the instances of Theorem 1, in which f is modular. □

Discussion. Theorems 1 and 2 show that the difficulty of MIMCS already lies in finding a feasible BMC containing the query; unless P = NP, neither feasibility nor any multiplicative approximation guarantee can be achieved in polynomial time. Since both proofs rely only on feasibility and never on the value of f, these results are independent of the influence estimate: they hold for every RR collection $\mathcal{J}$ and equally for the expected-influence objective. We therefore focus on exact search: Section 4 establishes a duplicate-free query-anchored enumeration framework, and Section 5 develops MIMC-B&B on top of it.

4 BASELINE ALGORITHM

In this section, we present Exhaustive Search, an exact baseline for MIMCS that also establishes the enumeration framework used by MIMC-B&B. Rather than enumerating arbitrary vertex subsets, it restricts the search domain structurally and enumerates query-anchored communities in a fixed order without duplication.

Relational-core preprocessing. We first iteratively remove from $\mathcal{H}[V_S]$ any vertex v that has fewer than k type-A neighbors for some applicable relational constraint $\langle \psi(v), A, k\rangle$; this includes self-type constraints $\langle A, A, k\rangle$, for which the condition counts the remaining type-A neighbors of a type-A vertex, as in k-core peeling. Since typed degrees can only decrease as vertices are removed, any such vertex cannot belong to an r-com. The process terminates with the *relational core* K. If $Q_0 \nsubseteq K$, or the query vertices do not lie in the same connected component of $\mathcal{H}[K]$, the instance is infeasible; otherwise, the search is restricted to the component containing Q_0.

Query-anchored enumeration. A recursive state is a pair (Q, C) with $Q_0 \subseteq Q \subseteq C \subseteq K$, where Q is the set of selected vertices and C is the residual domain; the state represents every completion Y with $Q \subseteq Y \subseteq C$. Starting from the single query $Q_0 = \{q\}$, Exhaustive Search expands Q only through its boundary $\partial_C(Q) = \{v \in C \setminus Q \mid \mathcal{N}_{\mathcal{H}}(v) \cap Q \neq \emptyset\}$, which keeps Q connected throughout the enumeration. Whenever Q itself forms a BMC, its influence is compared with the current best feasible solution $Q^\star$, referred to as the *incumbent*. The boundary vertices are processed in a fixed global order: after the child containing a vertex w is explored, w is excluded from the residual domain before the subsequent children are generated. This partitions the completions of the current state by their first selected boundary vertex, so that no completion is generated twice.

Exclusion propagation. Removing a vertex from the residual domain C may leave other vertices without the typed neighbors they need. We therefore repeatedly remove from $C \setminus Q$ every vertex w with $|\mathcal{N}_{\mathcal{H}}(w, A) \cap C| < k$ for some applicable constraint $\langle \psi(w), A, k\rangle$ until no such vertex remains; if a vertex of Q is reached, the state has no feasible completion and is discarded.

Algorithm 1 presents Exhaustive Search. Lines 1–3 perform the relational-core preprocessing, reject a query that is removed by it or already exceeds the budget, and restrict the search domain to the query-containing component; lines 4–6 run the recursive enumeration from Q_0 and return the incumbent or report infeasibility. Each recursive call checks the budget, updates the incumbent when Q is a BMC with larger influence (lines 8–9), and expands Q through its boundary in the fixed global order with the include-then-exclude partition described above (lines 10–13).

Theorem 3 (Exactness of Exhaustive Search). *Exhaustive Search enumerates every BMC containing Q_0 exactly once and therefore returns one with maximum influence f.*

Algorithm 1: Exhaustive Search

Input: HIN $\mathcal{H}$, constraints S, budget b, query $Q_0 = \{q\}$, RR collection $\mathcal{J}$
Output: An optimal community $Q^\star \supseteq Q_0$

1 $K \leftarrow$ relational core of $\mathcal{H}[V_S]$;
2 **if** $Q_0 \nsubseteq K$ ***or*** $c(Q_0) > b$ **then return** *infeasible*;
3 $C_0 \leftarrow$ residual domain of the query-containing component of $\mathcal{H}[K]$;
4 $Q^\star \leftarrow \perp$; Enum(Q_0, C_0);
5 **if** $Q^\star = \perp$ **then return** *infeasible*;
6 **return** $Q^\star$;
7 **Function** Enum(Q, C):
8   **if** $c(Q) > b$ **then return**;
9   **if** *Q is a BMC* ***and*** *($Q^\star = \perp$* ***or*** *$f(Q) > f(Q^\star)$)* **then** $Q^\star \leftarrow Q$;
10   $B \leftarrow \partial_C(Q)$, the boundary vertices of Q in C, in the fixed global order;
11   **foreach** $w \in B$ **do**
12   Enum$(Q \cup \{w\}, C)$;
13   exclude w from C and apply exclusion propagation;
14   **end**

Proof. Let Y be any BMC containing Q_0. Relational-core preprocessing preserves every vertex of Y: if a vertex of Y were the first such vertex removed, all other vertices of Y would still remain at that moment, contradicting that the removed vertex already has within Y all typed neighbors required by its relational constraints. For the same reason, the exclusion propagation during the search never removes a vertex of Y, since it removes only vertices whose typed support within the residual domain is insufficient, whereas the support of every vertex of Y lies within $Y \subseteq C$. Moreover, because all vertex costs are positive and $c(Y) \leq b$, every partial selection $Q \subseteq Y$ also satisfies the budget.

Consider a recursive state (Q, C) with $Q \subseteq Y \subseteq C$. If $Q = Y$, Y is evaluated at the current state. Otherwise, since Y is connected, at least one vertex of $Y \setminus Q$ belongs to $\partial_C(Q)$. Let w be the first such vertex in the fixed boundary order. Every boundary vertex preceding w lies outside Y, so excluding it preserves Y, while the child containing w still represents Y. After this child has been explored, excluding w prevents Y from appearing in any later sibling. Repeating this argument yields exactly one root-to-state path for Y. Hence every BMC containing Q_0 is enumerated exactly once, and retaining the largest f returns an optimum. □

Limitations. In the worst case the recursion visits $O(2^{|C_0|})$ states, since along any root-to-state path each vertex of the initial residual domain C_0 is either included in Q or excluded from C; each state is processed in time polynomial in $|C_0|$ and $|\mathcal{J}|$, so the worst-case running time is exponential, in line with Theorem 1. Although Exhaustive Search avoids duplicate enumeration, it still explores many states that cannot lead to a better solution. In particular, it does not estimate the minimum cost required to satisfy the unresolved relational constraints, and therefore cannot detect early whether a state can still be completed within the remaining budget. Even when such a completion is affordable, the algorithm has no upper bound on the influence attainable from the state. Finally, it branches over the entire boundary without exploiting which requirements are most constrained. These limitations motivate MIMC-B&B.

5 ADVANCED ALGORITHM

We now develop MIMC-B&B, which addresses these limitations by reasoning about the cost of completing a partial community.

Search state. MIMC-B&B uses the same recursive state (Q, C) as Exhaustive Search. For a vertex set $X \subseteq V$, let $\mathcal{N}_X(u, A) = \mathcal{N}_{\mathcal{H}}(u, A) \cap X$ denote the type-A neighbors of u within X. For a selected vertex $u \in Q$ and an applicable constraint $\langle \psi(u), A, k \rangle$, let $\delta_Q(u, A) = \max\{0, k - |\mathcal{N}_Q(u, A)|\}$ denote the number of additional type-A neighbors that u still requires, and let $P_C(u, A) = \mathcal{N}_{C \setminus Q}(u, A)$ be its remaining *supporter pool*. We call (u, A) an *open deficit* if $\delta_Q(u, A) > 0$, and let $\beta(Q) = b - c(Q)$ be the *residual budget*.

An open deficit does not by itself make the state infeasible, since it may still be repaired using vertices in its supporter pool; the following subsections quantify the cost of such repairs.

Example 2. Consider the HIN in Figure 1 under the constraints and costs of Example 1, and the state (Q, C) with $Q = \{a_1, p_1\}$ and $c(Q) = 5$, where C contains all authors, papers, and venues (topics lie outside V_S). Under $\langle A, P, 2 \rangle$, vertex a_1 has only one selected paper, so $\delta_Q(a_1, P) = 1$ with $P_C(a_1, P) = \{p_2, p_4\}$. Similarly, p_1 still requires one author and one venue, giving $\delta_Q(p_1, A) = 1$ with $P_C(p_1, A) = \{a_2, a_5\}$ and $\delta_Q(p_1, V) = 1$ with $P_C(p_1, V) = \{v_1\}$. Thus, the state has three open deficits, and $\beta(Q) = 15$ for $b = 20$.

5.1 Completion-Cost Reduction

A state may have enough remaining supporters to repair every open deficit but still require more cost than the residual budget allows. We therefore derive a lower bound on the additional cost incurred by any feasible completion of (Q, C). If this lower bound exceeds $\beta(Q)$, the state can be safely pruned.

Forced supporters. Consider an open deficit (u, A). If $|P_C(u, A)| < \delta_Q(u, A)$, no completion represented by (Q, C) can satisfy the corresponding constraint, and the state is infeasible. If $|P_C(u, A)| = \delta_Q(u, A)$, every vertex in $P_C(u, A)$ must belong to any feasible completion and can therefore be included in Q. The resulting changes may expose further forced inclusions or infeasibility, so this rule is applied repeatedly until no such deficit remains.

Direct completion cost. Let $\mathrm{minsum}_r(X)$ denote the sum of the r smallest vertex costs in X, or ∞ if $|X| < r$. For a selected vertex u, each open deficit (u, A) requires at least $\delta_Q(u, A)$ vertices from $P_C(u, A)$. Since supporter pools of different types are disjoint, the costs of the deficits of the same u can be added. Across different selected vertices, however, one supporter may serve several deficits and must not be charged repeatedly, so we take the maximum over the selected vertices rather than their sum: $LB^{(0)}(Q, C) = \max_{u \in Q} \sum_{A: \delta_Q(u,A) > 0} \mathrm{minsum}_{\delta_Q(u,A)}(P_C(u, A))$. Thus, $LB^{(0)}$ equals the largest cost of repairing the deficits of a single selected vertex; it remains valid even when supporters are shared across vertices.

Sharing-aware completion cost. The bound above considers one selected vertex at a time and may be loose when many vertices have unresolved requirements. We therefore aggregate the current demand by type while allowing a supporter to serve several selected vertices. For each demanded type A, let $D_A = \sum_{u \in Q} \delta_Q(u, A)$ be the total unresolved demand, counted in units of one required neighbor, and let $P_A = \bigcup_{u: \delta_Q(u,A) > 0} P_C(u, A)$ be its supporter pool. Let $\kappa_A = \max_{w \in P_A} \left|\{u \in Q \mid \delta_Q(u, A) > 0,\ w \in P_C(u, A)\}\right|$ denote the maximum number of these demand units that a single vertex in P_A can satisfy, since w can serve as one required neighbor of each selected vertex adjacent to it. Then every feasible completion requires at least $n_A = \lceil D_A / \kappa_A \rceil$ type-A supporters. Since supporters of different types are distinct, the costs of different types add up, which gives the first-level completion bound $LB^{(1)}(Q, C) = \sum_A \mathrm{minsum}_{n_A}(P_A)$.

Induced obligations. Satisfying the current deficits may create new ones: every supporter added to Q becomes a community member and must itself satisfy the relational constraints of its type. Consider a constraint $\langle A, B, k_{A,B} \rangle$. Since the identities of the n_A required type-A supporters are not yet known, we optimistically let $\rho_{\min}(A \to B) = \min_{w \in P_A} \max\{0, k_{A,B} - |\mathcal{N}_Q(w, B)|\}$ be the smallest type-B deficit that any such supporter could incur. The n_A supporters therefore induce at least $n_A \rho_{\min}(A \to B)$ type-B demand units. Summing over all demanded types A with a constraint on B gives the total induced type-B demand I_B.

Let $P^{(1)} = \bigcup_A P_A$ be the union of the first-level pools, and let $\kappa'_B = \max_w |\mathcal{N}_{\mathcal{H}}(w) \cap P^{(1)}|$, taken over the remaining type-B candidates $w \in C \setminus Q$, be the maximum number of these induced demand units that one such candidate can satisfy, since each potential supporter adjacent to w can charge w with at most one unit. If $I_B > 0$ but $\kappa'_B = 0$, the state is infeasible. Otherwise, at least $\lceil I_B / \kappa'_B \rceil$ type-B supporters are required. The n_B supporters already required by the current type-B deficits may also serve these induced obligations. Crediting this reuse optimistically, the number of *additional* type-B supporters that remains unavoidable is $m_B = \max\{0, \lceil I_B / \kappa'_B \rceil - n_B\}$.

Joint completion cost. The n_B direct supporters and the m_B additional supporters of type B are drawn from overlapping pools, so we price the two roles jointly rather than separately. Let $P_B^{(2)}$ be the remaining type-B candidates that can support at least one induced obligation. We choose n_B direct supporters from P_B together with m_B additional supporters from $P_B^{(2)}$ without selecting the same vertex twice, and let L_B^{JA} denote the minimum total cost of such a joint assignment. If no such choice exists, the state is infeasible. Summing over vertex types gives $LB^{(2)}(Q, C) = \sum_B L_B^{\mathrm{JA}}$. Since $L_B^{\mathrm{JA}} \geq \mathrm{minsum}_{n_B}(P_B)$ for every type B, $LB^{(2)}$ subsumes the first-level bound $LB^{(1)}$; it does not subsume $LB^{(0)}$, since the deficits $\delta_Q(u, A)$ of a single selected vertex can exceed the shared requirement n_A and its pools $P_C(u, A)$ are smaller than P_A. We therefore combine the two incomparable bounds into $LB_{\mathrm{comp}}(Q, C) = \max\{LB^{(0)}(Q, C), LB^{(2)}(Q, C)\}$.

Proposition 2 (Completion-Cost Lower Bound). *For every feasible completion Y of a state (Q, C), $c(Y \setminus Q) \geq LB_{\mathrm{comp}}(Q, C)$.*

Proof. Let Y be a feasible completion of (Q, C) and $R = Y \setminus Q$. We show $c(R) \geq LB^{(0)}(Q, C)$ and $c(R) \geq LB^{(2)}(Q, C)$ separately. For $LB^{(0)}$, fix any $u \in Q$: each open deficit (u, A) requires at least $\delta_Q(u, A)$ vertices of R from $P_C(u, A)$, and the pools of distinct types are disjoint, so $c(R) \geq \sum_{A: \delta_Q(u,A) > 0} \mathrm{minsum}_{\delta_Q(u,A)}(P_C(u, A))$; taking the maximum over $u \in Q$ preserves validity because each bound is charged to a single selected vertex. For $LB^{(2)}$, fix a type B. By the counting above, R contains at least n_B vertices of P_B and, if $m_B > 0$, at least $n_B + m_B$ vertices of $P_B^{(2)}$. Choose any n_B vertices of $R \cap P_B$ for the direct role; at least m_B vertices of $R \cap P_B^{(2)}$ remain unchosen and serve the induced role. This is a feasible joint assignment, so $L_B^{\mathrm{JA}} \leq c(R \cap (P_B \cup P_B^{(2)}))$, and summing over the disjoint types gives $LB^{(2)}(Q, C) \leq c(R)$. $\square$

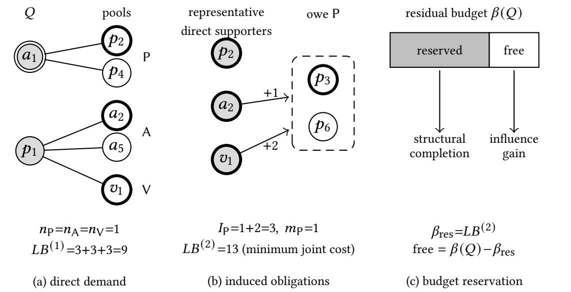


Figure 2: Completion-aware reasoning

Consequently, if $LB_{\text{comp}}(Q,C) > \beta(Q)$, no budget-feasible completion of (Q,C) exists, and the state can be safely pruned.

Example 3. Consider the state of Example 2 with the budget reduced from $b = 20$ to $b = 17$ (Figure 2), so that $\beta(Q) = 12$. The current deficits require at least one paper, one author, and one venue, giving $n_P = n_A = n_V = 1$ and $LB^{(1)} = 9$, which does not rule out a feasible completion. These first-level supporters, however, incur relational obligations of their own: under the optimistic induced-demand calculation they create three type-P demand units, and since one paper satisfies at most two of them and one paper is already required by the current deficits, at least one additional paper is necessary, i.e., $m_P = 1$. Jointly pricing the direct and induced supporter roles gives $LB^{(2)} = 13$ and $LB_{\text{comp}} = 13 > \beta(Q) = 12$. The state is therefore pruned.

5.2 Completion-Aware Influence Bounding

The completion-cost analysis above determines whether a state can still be completed within budget. When it can, the same analysis reveals that part of the residual budget is already committed to restoring relational feasibility. Treating this cost as freely available for influence therefore yields a loose upper bound. We exploit this observation to bound the influence attainable from (Q,C).

Fractional influence bound. For $v \in C \setminus Q$, let $\Delta_Q(v) = f(Q \cup \{v\}) - f(Q)$ denote its marginal influence gain. Ignoring relational completion, the entire residual budget $\beta(Q)$ could be spent on these gains. Allowing fractional vertex selection gives $UB_{\text{frac}}(Q,C) = f(Q)+FK(\beta(Q))$, where $FK(x)$ is the optimal value of the fractional knapsack whose items are the candidates $v \in C\setminus Q$ with profit $\Delta_Q(v)$ and weight $c(v)$, under capacity x. By submodularity, the sum of the marginal gains at Q upper-bounds the gain of any jointly selected set, so UB_{frac} is valid. It can nevertheless be loose because it ignores the cost required to complete Q.

Completion-aware bound. The analysis in Section 5.1 shows that every feasible completion must spend at least $LB^{(2)}(Q,C)$ on the direct and induced supporter roles. These supporters may themselves contribute influence, so their gain must also be credited. Among all joint supporter choices satisfying the n_B direct and m_B additional roles defined above, let g_{res} be the maximum sum of their marginal gains $\Delta_Q(v)$. After reserving the unavoidable completion cost, at most $\beta(Q) - LB^{(2)}(Q,C)$ of the budget remains freely available. We therefore obtain $UB_{\text{CA}}(Q,C) = f(Q) + g_{\text{res}} + FK(\beta(Q) - LB^{(2)}(Q,C))$. Figure 2(c) illustrates this budget reservation.

Lemma 1 (Completion-Aware Influence Bound). *For every feasible completion Y of (Q,C), $f(Y) \le UB_{\text{CA}}(Q,C)$.*

Proof. Let Y be a feasible completion and $R = Y \setminus Q$. By the extraction argument in the proof of Proposition 2, R contains a joint supporter set S realizing the n_B direct and m_B induced roles of every type B; hence $c(S) \ge LB^{(2)}(Q,C)$, and $\sum_{v\in S} \Delta_Q(v) \le g_{\text{res}}$ by the definition of g_{res}. The remaining vertices satisfy $c(R \setminus S) \le \beta(Q)-LB^{(2)}(Q,C)$. By submodularity, $f(Y)-f(Q) \le \sum_{v\in S} \Delta_Q(v)+\sum_{v\in R\setminus S} \Delta_Q(v) \le g_{\text{res}} + FK(\beta(Q) - LB^{(2)}(Q,C))$. ☐

Support-constrained fractional bound. The fractional bound can also be tightened by enforcing the supporter counts implied by the current deficits. Every feasible completion contains at least n_A vertices from P_A for each demanded type A. We therefore add these conditions to the fractional-knapsack relaxation:

$$\begin{aligned} UB_{\text{SC}}(Q,C) = f(Q) + \max\ & \textstyle\sum_{v\in C\setminus Q} \Delta_Q(v)x_v \\ \text{s.t.}\ & \textstyle\sum_{v\in C\setminus Q} c(v)x_v \le \beta(Q), \quad 0 \le x_v \le 1, \\ & \textstyle\sum_{v\in P_A} x_v \ge n_A \ \text{ for each demanded type } A. \end{aligned}$$

Lemma 2 (Support-Constrained Influence Bound). *For every feasible completion Y of (Q,C), $f(Y) \le UB_{\text{SC}}(Q,C)$.*

Proof. Let Y be a feasible completion, and set $x_v = 1$ for $v \in Y \setminus Q$ and $x_v = 0$ otherwise. Since $c(Y \setminus Q) \le \beta(Q)$ and Y contains at least n_A vertices of P_A for each demanded type A (Section 5.1), x is feasible for the relaxation, so $\sum_v \Delta_Q(v)x_v$ is at most its optimum. By submodularity, $f(Y) - f(Q) \le \sum_{v\in Y\setminus Q} \Delta_Q(v) = \sum_v \Delta_Q(v)x_v$. Hence $f(Y) \le UB_{\text{SC}}(Q,C)$. ☐

The two bounds exploit complementary information, so MIMC-B&B uses $UB(Q,C) = \min\{UB_{\text{CA}}(Q,C), UB_{\text{SC}}(Q,C)\}$. During search, a state is pruned whenever $Q^\star \ne \perp$ and $UB(Q,C) \le f(Q^\star)$, since no feasible completion can improve the incumbent.

Computation. All quantities above are computed from the current state in time near-linear in the size of the residual domain. Scanning the neighborhoods of the selected vertices and of the first-level pools yields the deficits, pools, κ_A, $\rho_{\min}$, and κ'_B in $O(\sum_{v\in C} d(v))$ time, and minsum_r is an order statistic obtained by linear-time selection. L_B^{JA} is computed exactly by enumerating how many candidates of $P_B \cap P_B^{(2)}$ serve each role and pricing the rest of each role by minsum over $P_B \setminus P_B^{(2)}$ and $P_B^{(2)} \setminus P_B$, in $O(p_B \log p_B + n_B m_B)$ time with $p_B = |P_B \cup P_B^{(2)}|$. FK sorts the candidates by $\Delta_Q(v)/c(v)$ and fills the capacity greedily in $O(|C| \log |C|)$ time once the marginal gains have been evaluated on the RR collection. UB_{SC} needs no general-purpose solver: charging the budget at a price $\lambda \ge 0$, i.e., replacing the budget constraint by the penalty $\lambda(\beta(Q) - \sum_v c(v)x_v)$, makes the relaxation separable over the disjoint pools and solvable greedily; every price yields a valid upper bound, and since the bound is convex and piecewise linear in λ, ternary search over the $O(|C|)$ candidate prices $\Delta_Q(v)/c(v)$ minimizes it in $O(|C| \log |C|)$ time; the result may exceed the optimum of the relaxation but remains valid.

Example 4. Consider the state $Q = \{a_1, a_3, p_1, p_2, p_3, v_1\}$ under the constraints and costs of Example 1 with budget $b = 20$. Its current cost is $c(Q) = 17$, leaving $\beta(Q) = 3$, and $f(Q) = 7$. If relational completion is ignored, the fractional bound may spend all three remaining cost units on influence. Candidate a_6 costs 4 and covers one additional RR set, so the relaxation credits three quarters of this gain and gives $UB_{\text{frac}} = 7.75$. The completion analysis, however, shows that the entire residual budget is required to satisfy the unresolved relational constraints, i.e., $LB^{(2)} = 3$. The corresponding supporter roles add no new RR coverage in this state, so $g_{\text{res}} = 0$, leaving no budget for unrestricted influence gain. Hence $UB_{\text{CA}} = 7$, which is tight at this state.

5.3 Branching Strategy

After reduction and bounding, the remaining feasible completions must be partitioned into recursive branches. Instead of branching over the entire boundary as in Exhaustive Search, MIMC-B&B exploits unresolved relational deficits to identify a smaller set that every feasible completion must intersect.

Deficit-guided branch selection. For an open deficit (u, A), define its *supporter slack* as $s_Q(u, A) = |P_C(u, A)| - \delta_Q(u, A)$. A smaller slack means that fewer alternatives remain for satisfying the requirement. MIMC-B&B therefore selects an open deficit with minimum supporter slack and uses its supporter pool $B = P_C(u, A)$ as the branch set. Ties are resolved deterministically. Since $P_C(u, A) \subseteq \partial_C(Q)$, replacing boundary expansion by deficit-guided branching preserves the connectivity invariant of Section 4.

Since the selected deficit is open, every feasible completion represented by (Q, C) must contain at least one vertex of B. The vertices of B are processed sequentially: after the branch containing a vertex v is explored, v is excluded from the residual domain before the next branch is generated. Consequently, every feasible completion is assigned to the branch of its first selected vertex in B, so the branches are complete and mutually exclusive. If no open deficit remains, MIMC-B&B falls back to the boundary expansion of Exhaustive Search (Section 4).

Candidate ordering. Within the selected branch set B, MIMC-B&B visits candidates in descending order of marginal influence gain per unit cost, $\Delta_Q(v)/c(v)$. This favors candidates that can improve the incumbent substantially with limited budget consumption. A stronger incumbent can in turn make the influence upper bound more effective in pruning subsequent states. This ordering changes only the order in which the branches are explored and does not affect the branch partition or exactness.

5.4 Overall Algorithm and Exactness

Algorithm 2 summarizes MIMC-B&B, which adds the reduction, bounding, and branching steps of Sections 5.1–5.3 to the enumeration framework of Algorithm 1.

MIMC-B&B first computes the relational core and restricts the search to the query-containing component (lines 1–3). A verified initial feasible community, when available, initializes the incumbent before the exact search begins (line 4). At each recursive state, forced consequences are propagated first, and states that are already structurally or budget infeasible are discarded (lines 9–10). The incumbent is updated whenever the current selection forms a better BMC (line 11). The completion-cost lower bound and influence upper bound then prune states that cannot yield an affordable or better completion, respectively (lines 12–13). Otherwise, the algorithm selects a branch set using the deficit-guided strategy and explores its candidates in gain-per-cost order using the include-then-exclude partition described in Section 5.3 (lines 14–19).

Algorithm 2: MIMC-B&B

Input: HIN $\mathcal{H}$, constraints S, budget b, query $Q_0 = \{q\}$, RR collection $\mathcal{J}$
Output: An optimal community $Q^\star \supseteq Q_0$, or infeasible

1 $K \leftarrow$ relational core of $\mathcal{H}[V_S]$;
2 **if** $Q_0 \not\subseteq K$ **or** $c(Q_0) > b$ **then return** *infeasible*;
3 $C_0 \leftarrow$ residual domain of the query-containing component of $\mathcal{H}[K]$;
4 $Q^\star \leftarrow$ a verified initial feasible community, or $\perp$ if none;
5 `Search`(Q_0, C_0);
6 **if** $Q^\star = \perp$ **then return** *infeasible*;
7 **return** $Q^\star$;
8 **Function** `Search`(Q, C):
  `// reduction: propagation (Section 5.1)`
9   apply forced-supporter and exclusion propagation to a fixpoint;
10   **if** *some supporter pool is insufficient* ***or*** $c(Q) > b$ **then return**;
11   **if** *Q is a BMC* ***and*** *($Q^\star$=$\perp$* ***or*** *$f(Q) > f(Q^\star)$)* **then** $Q^\star \leftarrow Q$;
  `// reduction: completion-cost pruning (Section 5.1)`
12   **if** $LB_{\text{comp}}(Q, C) > \beta(Q)$ **then return**;
  `// bounding (Section 5.2)`
13   **if** $Q^\star \neq \perp$ ***and*** $UB(Q, C) \leq f(Q^\star)$ **then return**;
  `// deficit-guided branching (Section 5.3)`
14   $B \leftarrow$ branch set selected as in Section 5.3;
15   order the vertices of B by descending $\Delta_Q(v)/c(v)$;
16   **foreach** $v \in B$ *in this order* **do**
17   `Search`$(Q \cup \{v\}, C)$;
18   exclude v from C and apply exclusion propagation;
19   **end**

Initial feasible community. Before the exact search, we construct a feasible community containing q by a ladder of constructive procedures, each followed by verification: (i) the closure of q, a small vertex set containing q in which every member satisfies its constraints; (ii) a bounded-radius ball around q, peeled to its relational core with q protected and then minimized by removing expensive vertices whose removal preserves feasibility; (iii) closure-greedy growth that repeatedly adds the candidate whose closure yields the largest influence gain per unit cost within the budget; and (iv) a few randomized restarts of (ii) with deficit-driven repair. The first verified result initializes the incumbent $Q^\star$ and thereby strengthens pruning from the start without affecting exactness.

Theorem 4 (Exactness of MIMC-B&B). *MIMC-B&B returns an optimal BMC containing Q_0, or correctly reports infeasibility if no such BMC exists.*

Proof. By Theorem 3, the relational-core preprocessing and query-anchored include-then-exclude enumeration of Exhaustive Search preserve all feasible communities containing Q_0 and generate them without duplication, and its proof also covers exclusion propagation. It therefore remains to show that the operations added by MIMC-B&B do not remove any feasible completion that could improve the incumbent. Forced-supporter propagation is safe by the argument of Section 5.1: a forced supporter belongs to every feasible completion of the state, and a deficit with an insufficient pool admits none.

The remaining pruning operations do not remove an improving feasible completion. By Proposition 2, a state with $LB_{\text{comp}}(Q, C) > \beta(Q)$ has no budget-feasible completion. The influence bound $UB(Q, C) = \min\{UB_{\text{CA}}(Q, C), UB_{\text{SC}}(Q, C)\}$ upper-bounds the influence of every feasible completion, so a state with $UB(Q, C) \leq f(Q^\star)$ cannot improve the incumbent.

Table 3: Dataset statistics

Dataset	Vertices	Edges	Vertex types	Edge types
DBLP	881,039	1,366,161	4	3
IMDB	854,616	3,898,136	4	3
TMDB	71,978	113,581	7	6
DBpedia	5,900,558	17,641,286	413	11,519
PubMed	14,256	33,556	4	3

Finally, as shown in Section 5.3, every feasible completion intersects the deficit-guided branch set, and the sequential include-then-exclude processing assigns each completion to exactly one branch; without open deficits, MIMC-B&B falls back to the duplicate-free boundary expansion of Exhaustive Search. Hence branching preserves every feasible completion and enumerates it at most once.

Therefore, every feasible community that can improve the incumbent is either examined or contained in a safely pruned state. The final incumbent is consequently an optimal BMC containing Q_0; if no incumbent is found, no feasible BMC containing Q_0 exists. □

As stated in Section 3, exactness refers to the optimization of f on the fixed RR collection.

Complexity. The preprocessing, reduction, bounding, and branching operations at each search state are polynomial in the size of the residual domain. The number of recursive states, however, remains exponential in the worst case, as expected from the NP-hardness of MIMCS (Theorem 1). MIMC-B&B therefore improves practical efficiency by pruning infeasible or noncompetitive states and by exploring promising branches earlier, without changing the worst-case exponential complexity.

6 EXPERIMENTAL EVALUATION

In this section, we evaluate the effectiveness of MIMCS and the efficiency of MIMC-B&B: after the setup (Section 6.1), we present a case study (Section 6.2), evaluate efficiency, the optimization techniques, and the number of vertex types (Section 6.3), and examine scalability (Section 6.4).

6.1 Experimental Setup

Datasets. Table 3 summarizes the five real-world HINs used: DBLP [20, 35] (bibliographic), IMDB [14, 42] and TMDB [12] (movies), DBpedia [19] (a knowledge graph with hundreds of vertex types), and PubMed [40] (biomedical). All graphs are treated as undirected with duplicate edges removed.

Experimental settings. The main efficiency and ablation experiments use three vertex types per dataset: we randomly select the participating types, designate one of them as the target type, randomly generate relational constraints over them, and keep only settings that admit at least one r-com. Meta-paths are schema-valid paths of length two, namely one symmetric target-to-target meta-path and asymmetric meta-paths from the non-target types to the target type. Query vertices are sampled from target-type vertices that belong to an r-com under the setting; once generated, settings and queries are fixed and reused across the compared algorithms and parameter values. We use three settings each on DBLP, IMDB, and PubMed, five each on TMDB and DBpedia, and five query vertices per setting, except for two DBLP settings whose target types admit only two eligible queries.

Parameters. Each vertex v is assigned cost $c(v) = 1 + \ln(1 + d(v))$, so higher-degree vertices cost more. The constraint tightness γ, the average threshold k of the relational constraints, is varied over $\{2, 3, 4, 5, 6\}$ with default 4, and the budget over five dataset-specific levels, $\{100, 115, 130, 145, 160\}$ on DBLP, $\{60, 95, 130, 165, 200\}$ on IMDB, $\{95, 110, 125, 140, 155\}$ on TMDB, $\{45, 65, 85, 105, 125\}$ on DBpedia, and $\{150, 165, 180, 195, 210\}$ on PubMed, with the middle level as the default; each experiment varies one parameter while fixing the other at its default.

RR-set generation. Following Section 3, the target graph G_T connects two target vertices if an instance of $\mathcal{P}_s$ joins them, an activation attempt from u to v succeeds with the WC probability $1/d_{G_T}(v)$, and the entry set of a non-target vertex consists of the target vertices reached by instances of $\mathcal{P}_a$. An RR set is sampled by drawing a target vertex uniformly at random and collecting, by a reverse breadth-first traversal, the vertices that reach it in a random realization of this model. Each run samples 5,000 RR sets with random seed 1 before the search starts, so that all algorithms, queries, and parameter values of a setting use the identical collection.

Algorithms. No existing method solves MIMCS (Sections 1 and 7), so we compare Exhaustive Search (Section 4) with MIMC-B&B (Section 5), the only exact references for MIMCS, isolate each optimization family by the variants w/o Reduction, w/o Bounding, and w/o Branching, and compare the returned communities with those of ICSH and SACH in Section 6.2.

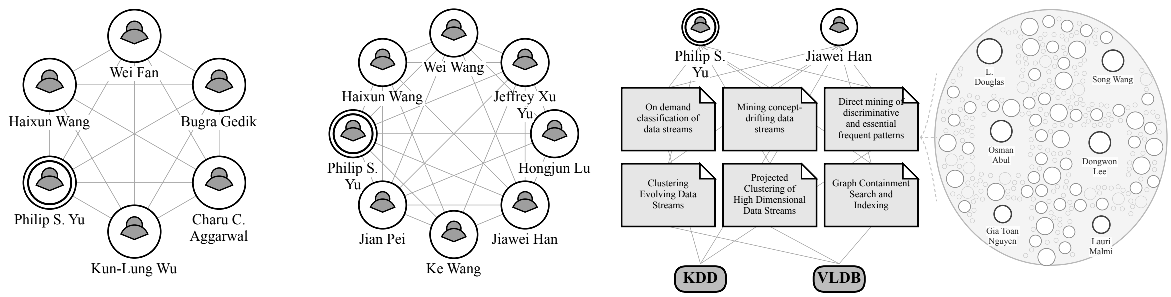
Metrics. Running time is the end-to-end time of a run; a run exceeding the one-hour limit is counted as 3600 seconds and plotted at the Inf mark. **Completion ratio** is the percentage of runs that, within the time limit, either certify an optimal feasible community or prove that none exists, including queries rejected by relational-core preprocessing; in the main experiments, 35.8%, 31.1%, 43.1%, 6.7%, and 16.3% of the DBLP, IMDB, TMDB, DBpedia, and PubMed instances, respectively, are proved infeasible. Results are averaged over the queries of each setting and then over settings.

Implementation. All algorithms are implemented in C++17, compiled with GCC -O3, and run single-threaded on OzSTAR cluster nodes with two 32-core AMD EPYC 7543 (2.8 GHz) CPUs and 256 GB of RAM; the peak resident memory of a run never exceeds 1.4 GB.

6.2 Effectiveness Evaluation

Effectiveness case study. We run the case study on a small DBLP network with 37,791 vertices (14,475 authors, 14,376 papers, 20 venues, and 8,920 topics) and 170,794 edges extracted from the original DBLP network, use Philip S. Yu as the common query author, and compare the communities returned by ICSH [47], SACH [26], and MIMCS in Figure 3. ICSH and SACH use the author–paper–author meta-path with $k = 5$ and return communities of six and eight authors, respectively. For MIMCS, we use unit vertex costs and budget $b = 10$ under the relational constraints $\langle A, P, 3\rangle$, $\langle P, A, 2\rangle$, $\langle V, P, 3\rangle$, and $\langle P, V, 1\rangle$; the optimal community contains two authors, six papers, and two venues.

Since the methods optimize different objectives, the scores in Figures 3(a) and (b) are rescaled only for readability. Evaluated under our influence model on the same RR collection, however, the ICSH and SACH communities reach an estimated 903 and 1,699 target vertices, respectively, versus 7,009 for MIMCS, and neither is



(Skyline vector: [2.563, 6.940]) (a) ICSH

(Significance: 1.157) (b) SACH

(Estimated reach: 7,000+ target vertices) (c) MIMCS

Figure 3: Case study on DBLP

a feasible BMC under the case-study constraints, since both consist of authors only and every member violates $\langle A, P, 3\rangle$. ICSH and SACH return influential or cohesive researchers around the query and report an aggregate score, whereas MIMCS returns the papers and venues that jointly support the selected researchers under the cross-type constraints, and its RR-based propagation view exposes the target vertices through which the estimated influence is realized (Figure 3(c)). Heterogeneous entities are thus first-class community members rather than auxiliary connectors, and collective influence is traced to its estimated propagation reach.

6.3 Efficiency Evaluation

We first compare MIMC-B&B with Exhaustive Search under varying budgets and relational constraints, then isolate the contribution of each optimization family, and finally examine how the number of vertex types affects exact search.

Effect of budget. MIMC-B&B consistently outperforms Exhaustive Search on DBLP, IMDB, TMDB, and DBpedia, most clearly on DBLP and DBpedia (Figures 4(a)–(e)); a larger budget widens the gap because it admits more feasible combinations that Exhaustive Search must enumerate, whereas completion-aware reduction and bounding eliminate most of these states, so MIMC-B&B also achieves higher completion ratios in the difficult settings. PubMed remains hard for both: under large budgets many instances approach the one-hour limit, so pruning reduces but does not remove the combinatorial difficulty.

Effect of relational constraints. Figures 4(f)–(j) show a non-monotone relationship between constraint tightness and search cost: a larger γ imposes stronger completion requirements, which can make a feasible community harder to construct, but it also removes structurally invalid candidates earlier and strengthens the pruning of MIMC-B&B, so intermediate levels can be harder than either loose or tight settings, as on DBLP, IMDB, and TMDB, while DBpedia stays easy across the sweep and PubMed becomes easier only under the tighter constraints. MIMC-B&B keeps a clear advantage in the difficult regimes, indicating that relational constraints provide structural information for exact pruning rather than merely restricting the feasible solutions. Under the tightest constraints, many query vertices leave the relational core, so these instances are certified infeasible during preprocessing and the completion ratios of both algorithms rise.

Ablation study. Figure 5 compares MIMC-B&B with w/o Reduction, w/o Bounding, and w/o Branching, each of which preserves exactness while disabling one optimization family, under the budget (top row) and constraint-tightness (bottom row) sweeps as above, with Exhaustive Search as the unoptimized reference.

The three families are complementary. Removing bounding causes some of the largest slowdowns, particularly on DBpedia, which shows the importance of ruling out states whose remaining influence cannot improve the incumbent; reduction provides consistent gains across the regimes by eliminating states whose relational requirements cannot be completed within the remaining budget; and removing deficit-guided branching also increases the search effort, though less uniformly, because branching changes how the remaining search space is partitioned and interacts with the subsequent reduction and bounding. The full MIMC-B&B is therefore the most robust across both sweeps, and no single family accounts for its efficiency. Measured by the number of visited search states at the default setting, MIMC-B&B explores 5× (IMDB) to about 900× (DBLP) fewer states than Exhaustive Search, and on DBpedia its initial feasible community is certified optimal at the root, so the search ends after a single state; disabling bounding increases the number of visited states the most, by 2.5× (DBLP) to 18× (PubMed).

Effect of the number of vertex types. On DBLP, IMDB, TMDB, and PubMed, we fix the target type and query vertex and expand the participating type set from two to four types, with schema-valid constraints at $\gamma = 4$ and the default budget (Table 4). The search cost is not monotone in the number of types: additional types enlarge the candidate combinations but also add relational requirements that restrict feasible completions and strengthen pruning. On DBLP, IMDB, and TMDB the added constraints prove many instances infeasible almost immediately—on DBLP the average drops from 2732.8 s with a completion ratio of 30.0% at two types to 14.6 s with every instance completed at three—whereas on PubMed the gene, disease, and chemical vertices have typed degrees in the hundreds, so each added type enlarges the supporter pools far more than its constraints restrict them, and beyond two types only 8.3% of the instances complete.

6.4 Scalability Test

We vary each HIN from 20% to 100% by nested, type-stratified sampling: every sampled graph retains the validated structural backbone under $\gamma = 4$, the remaining vertices are progressively added from the full HIN, and the same query vertices, relational constraints, and budget are reused across scales. This design is also a

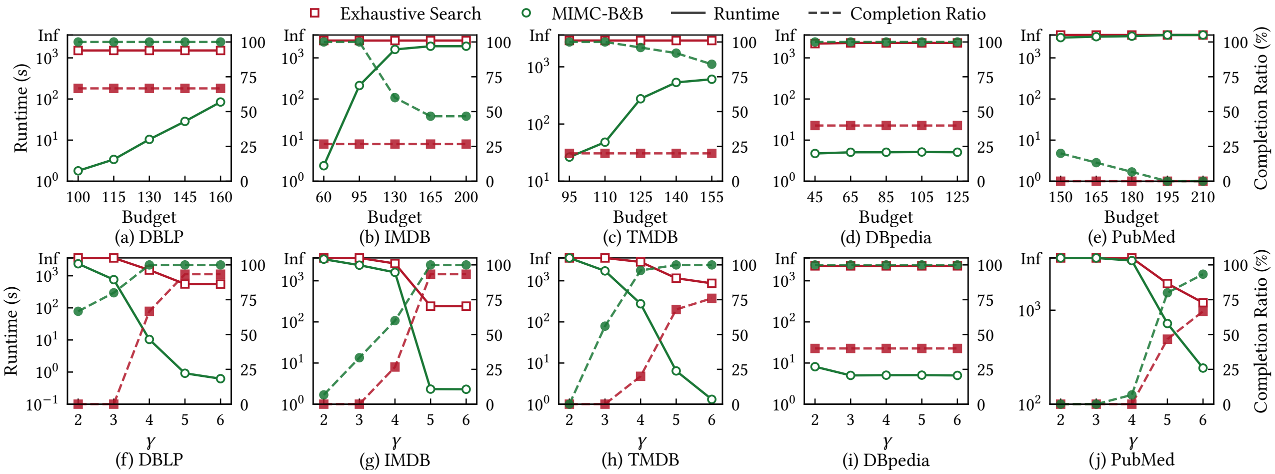


Figure 4: Efficiency with varying budget (top) and γ (bottom)

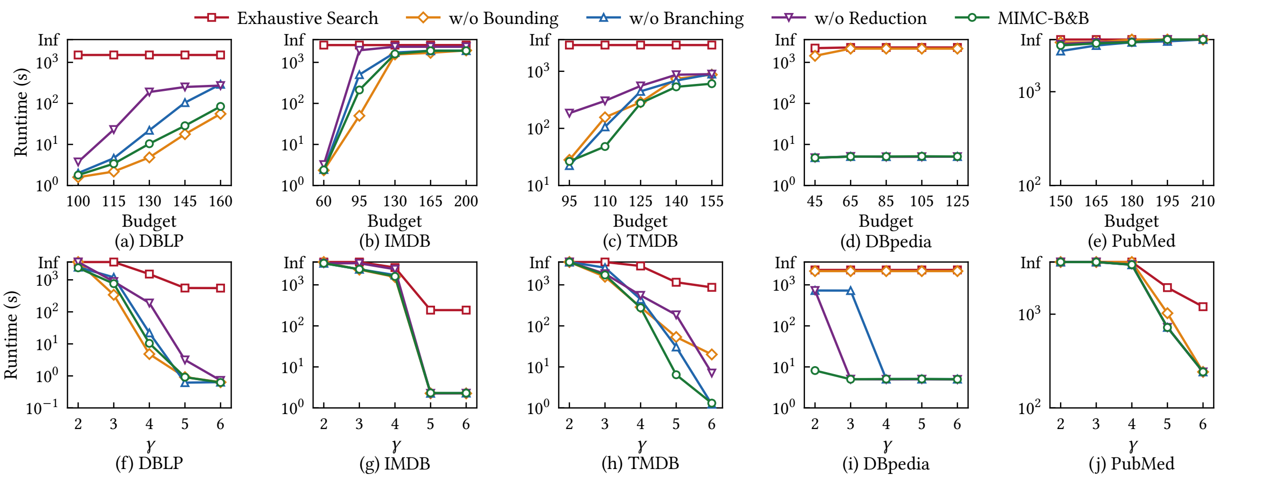


Figure 5: Ablation with varying budget (top) and γ (bottom)

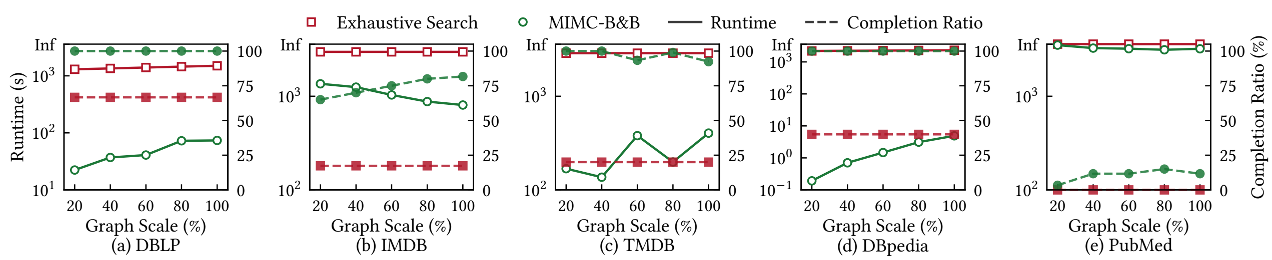


Figure 6: Scalability

Table 4: Varying the number of vertex types

	2 Types		3 Types		4 Types	
Dataset	Time (s)	Compl. (%)	Time (s)	Compl. (%)	Time (s)	Compl. (%)
DBLP	2732.8	30.0	14.6	100.0	403.7	90.0
IMDB	2211.0	43.3	731.2	83.3	4.1	100.0
TMDB	2830.8	21.7	345.7	96.7	9.2	100.0
PubMed	2401.4	33.3	3330.4	8.3	3302.6	8.3

limitation: the smaller graphs are not uniform subgraphs of the HIN, so the measured growth reflects the amount of surrounding data rather than the structural difficulty of the queries, which is governed by the size $|C_0|$ of the query's component of the relational core (26–19,112 on DBLP, 12–342,896 on IMDB, 15–15,840 on TMDB, 20–454 on DBpedia, and 37–4,820 on PubMed across the main experiments). Figure 6 shows that MIMC-B&B retains a clear advantage over Exhaustive Search as the HIN grows: on DBLP its mean running time increases from 22.3 s at 20% scale to 73.7 s on the full graph, and on DBpedia from 0.19 s to 4.9 s. The IMDB and TMDB curves are less monotone because the means depend on which difficult instances finish within the one-hour limit, and PubMed remains the hardest case, where both methods are largely time-limit dominated and Exhaustive Search completes no instance at any scale.

7 RELATED WORK

Community search. Community search retrieves cohesive subgraphs containing the query vertices, using structural models such as k-core and k-truss as well as attributed and size-constrained variants [6–8, 13, 31, 41, 43]. In HINs, meta-path-based models search communities of a designated type connected through prescribed semantic paths [10, 16], whereas relational community search imposes typed degree constraints on the selected vertices so that the community spans multiple types [3, 15]; Budgeted Strong Community Search adds a size constraint and optimizes the number of connecting meta-path instances [46]. MIMCS adopts the relational community model but optimizes collective propagation influence under heterogeneous vertex costs rather than cohesiveness or relationship strength.

Influential community search. Influential community search finds cohesive communities that are also influential. One line scores a community by predefined vertex weights or importance values, with extensions to top-k and progressive search, multiple importance dimensions, aggregation functions, personalized queries, and dynamic graphs [1, 23–25, 29, 33, 39], and over HINs by type-specific importance values (ICSH [47]) or the minimum member significance of a same-type meta-path community (SACH [26]); such member-level scores ignore the overlap among the selected members. Another line evaluates influence through propagation on social networks [22, 38, 44], which captures collective influence but considers neither multi-type members nor cross-type relational constraints. MIMCS evaluates a multi-type community by the propagation that its members jointly initiate via multiple meta-paths.

Influence maximization. Influence maximization selects k seeds maximizing the expected spread under the independent cascade or linear threshold model [4, 5, 17]; the spread is monotone and submodular, so greedy selection achieves a $(1-1/e)$-approximation [17], made near-linear by reverse reachable set sampling [2, 11, 36, 37]. Under a knapsack budget, greedy variants retain a constant-factor guarantee [18, 34]. MIMCS adopts the RR-set estimator, but its seeds are the target entries of a community that must satisfy relational constraints and a budget; since MIMCS admits no polynomial-time approximation (Theorem 2), the greedy guarantee does not apply.

Exact search for constrained communities. Exact branch-and-bound algorithms with structural reductions and degree-based bounds have been developed for community search problems whose feasible solutions are hard to construct, such as minimum k-core search [21, 45] and size-prescribed or size-bounded community search [27, 28, 43]. They operate on a single vertex type under a uniform size or degree requirement and optimize size or cohesiveness, whereas MIMC-B&B must resolve deficits across multiple types with heterogeneous costs and therefore prices relational completion jointly with the budget and bounds influence through completion-aware relaxations.

8 CONCLUSION

In this paper, we study MIMCS over HINs, which searches for a query-anchored multi-type community that maximizes collective propagation influence subject to relational constraints and a vertex-cost budget. We prove that MIMCS is NP-hard and admits no polynomial-time approximation unless P = NP, and we develop MIMC-B&B, an exact branch-and-bound algorithm that uses completion-aware reasoning for reduction and influence bounding, together with deficit-guided branching. Experiments on five real-world HINs show that MIMC-B&B substantially improves running time and completion ratio over Exhaustive Search across the tested settings, while the case study illustrates the effectiveness of the resulting multi-type communities. Future work includes exact search for multi-query anchors and larger type sets, tighter bounds that couple influence with relational completion, approximation algorithms under relaxed feasibility requirements, and the maintenance of communities over dynamic HINs.

REFERENCES

[1] Fei Bi, Lijun Chang, Xuemin Lin, and Wenjie Zhang. 2018. An optimal and progressive approach to online search of top-k influential communities. *Proceedings of the VLDB Endowment* 11, 9 (2018), 1056–1068.

[2] Christian Borgs, Michael Brautbar, Jennifer Chayes, and Brendan Lucier. 2014. Maximizing social influence in nearly optimal time. In *Proceedings of the twenty-fifth annual ACM-SIAM symposium on Discrete algorithms*. SIAM, 946–957.

[3] Guoxin Chen, Fangda Guo, Yongqing Wang, Yanghao Liu, Peiying Yu, Huawei Shen, and Xueqi Cheng. 2024. FCS-HGNN: Flexible multi-type community search in heterogeneous information networks. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management (CIKM)*. ACM, New York, NY, USA, 207–217.

[4] Wei Chen, Chi Wang, and Yajun Wang. 2010. Scalable influence maximization for prevalent viral marketing in large-scale social networks. In *Proceedings of the 16th ACM SIGKDD international conference on Knowledge discovery and data mining*. 1029–1038.

[5] Wei Chen, Yifei Yuan, and Li Zhang. 2010. Scalable Influence Maximization in Social Networks under the Linear Threshold Model. In *2010 IEEE International Conference on Data Mining*. 88–97.

[6] Wanyun Cui, Yanghua Xiao, Haixun Wang, and Wei Wang. 2014. Local search of communities in large graphs. In *Proceedings of the 2014 ACM SIGMOD International Conference on Management of Data (SIGMOD)*. ACM, New York, NY, USA, 991–1002.

[7] Yixiang Fang, Reynold Cheng, Siqiang Luo, and Jiafeng Hu. 2016. Effective community search for large attributed graphs. *Proceedings of the VLDB Endowment* 9, 12 (2016), 1233–1244.

[8] Yixiang Fang, Xin Huang, Lu Qin, Ying Zhang, Wenjie Zhang, Reynold Cheng, and Xuemin Lin. 2020. A survey of community search over big graphs. *The VLDB Journal* 29, 1 (2020), 353–392.

[9] Yixiang Fang, Kai Wang, Xuemin Lin, and Wenjie Zhang. 2022. *Cohesive Subgraph Search over Large Heterogeneous Information Networks*. Springer, Cham.

[10] Yixiang Fang, Yixing Yang, Wenjie Zhang, Xuemin Lin, and Xin Cao. 2020. Effective and efficient community search over large heterogeneous information networks. *Proceedings of the VLDB Endowment* 13, 6 (2020), 854–867.

[11] Qintian Guo, Sibo Wang, Zhewei Wei, and Ming Chen. 2020. Influence maximization revisited: Efficient reverse reachable set generation with bound tightened. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data (SIGMOD)*. ACM, New York, NY, USA, 2167–2181.

[12] Han Huang, Leilei Sun, Bowen Du, Chuanren Liu, Weifeng Lv, and Hui Xiong. 2021. Representation learning on knowledge graphs for node importance estimation. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*. ACM, 646–655.

[13] Xin Huang, Hong Cheng, Lu Qin, Wentao Tian, and Jeffrey Xu Yu. 2014. Querying k-truss community in large and dynamic graphs. In *Proceedings of the 2014 ACM SIGMOD International Conference on Management of Data (SIGMOD)*. ACM, New York, NY, USA, 1311–1322.

[14] IMDb. [n.d.]. IMDb non-commercial datasets. https://www.imdb.com/interfaces/. Accessed 2026.

[15] Xun Jian, Yue Wang, and Lei Chen. 2020. Effective and efficient relational community detection and search in large dynamic heterogeneous information networks. *Proceedings of the VLDB Endowment* 13, 10 (2020), 1723–1736.

[16] Yangqin Jiang, Yixiang Fang, Chenhao Ma, Xin Cao, and Chunshan Li. 2022. Effective community search over large star-schema heterogeneous information networks. *Proceedings of the VLDB Endowment* 15, 11 (2022), 2307–2320.

[17] David Kempe, Jon Kleinberg, and Éva Tardos. 2003. Maximizing the spread of influence through a social network. In *Proceedings of the ninth ACM SIGKDD international conference on Knowledge discovery and data mining*. 137–146.

[18] Samir Khuller, Anna Moss, and Joseph (Seffi) Naor. 1999. The budgeted maximum coverage problem. *Inform. Process. Lett.* 70, 1 (1999), 39–45.

[19] Jens Lehmann, Robert Isele, Max Jakob, Anja Jentzsch, Dimitris Kontokostas, Pablo N. Mendes, Sebastian Hellmann, Mohamed Morsey, Patrick van Kleef, Sören Auer, and Christian Bizer. 2015. DBpedia: A large-scale, multilingual knowledge base extracted from Wikipedia. *Semantic Web* 6, 2 (2015), 167–195.

[20] Michael Ley. 2002. The DBLP computer science bibliography: Evolution, research issues, perspectives. In *String Processing and Information Retrieval (SPIRE)*. Springer, Berlin, Heidelberg, 1–10.

[21] Conggai Li, Fan Zhang, Ying Zhang, Lu Qin, Wenjie Zhang, and Xuemin Lin. 2019. Efficient progressive minimum k-core search. *Proceedings of the VLDB Endowment* 13, 3 (2019), 362–375.

[22] Jianxin Li, Xinjue Wang, Ke Deng, Xiaochun Yang, Timos Sellis, and Jeffrey Xu Yu. 2017. Most influential community search over large social networks. In *2017 IEEE 33rd international conference on data engineering (ICDE)*. IEEE, 871–882.

[23] Rong-Hua Li, Lu Qin, Fanghua Ye, Jeffrey Xu Yu, Xiaokui Xiao, Nong Xiao, and Zibin Zheng. 2018. Skyline community search in multi-valued networks. In *Proceedings of the 2018 International Conference on Management of Data (SIGMOD)*. ACM, New York, NY, USA, 17.

[24] Rong-Hua Li, Lu Qin, Jeffrey Xu Yu, and Rui Mao. 2015. Influential community search in large networks. *Proceedings of the VLDB Endowment* 8, 5 (2015), 509–520.

[25] Rong-Hua Li, Lu Qin, Jeffrey Xu Yu, and Rui Mao. 2017. Finding influential communities in massive networks. *The VLDB Journal* 26, 6 (2017), 751–776.

[26] Yanghao Liu, Fangda Guo, Bingbing Xu, Peng Bao, Huawei Shen, and Xueqi Cheng. 2024. SACH: Significant-attributed community search in heterogeneous information networks. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE.

[27] Yang Liu, Hejiao Huang, Kaiqiang Yu, Shengxin Liu, Cheng Long, and Zhaoquan Gu. 2025. Efficient size-bounded community search, revisited: Frameworks for practical improvements. *Proceedings of the ACM on Management of Data* 3, 6 (2025), 1–26.

[28] Yiping Liu, Bo Yan, Bo Zhao, Hongyi Su, Yang Chen, and Michael Witbrock. 2023. Efficient size-prescribed k-core search. In *Proceedings of the 2023 IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining (ASONAM)*. ACM, New York, NY, USA, 271–275.

[29] You Peng, Song Bian, Rui Li, Sibo Wang, and Jeffrey Xu Yu. 2022. Finding top-r influential communities under aggregation functions. In *2022 IEEE 38th International Conference on Data Engineering (ICDE)*. IEEE, IEEE, 1941–1954.

[30] Chuan Shi, Yitong Li, Jiawei Zhang, Yizhou Sun, and Philip S. Yu. 2017. A survey of heterogeneous information network analysis. *IEEE Transactions on Knowledge and Data Engineering* 29, 1 (2017), 17–37.

[31] Mauro Sozio and Aristides Gionis. 2010. The community-search problem and how to plan a successful cocktail party. In *Proceedings of the 16th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*. ACM, New York, NY, USA, 939–948.

[32] Yizhou Sun, Jiawei Han, Xifeng Yan, Philip S. Yu, and Tianyi Wu. 2011. PathSim: Meta path-based top-k similarity search in heterogeneous information networks. *Proceedings of the VLDB Endowment* 4, 11 (2011), 992–1003.

[33] Youran Sun, Yingli Zhou, Yixiang Fang, Cheng Chen, Yongmin Hu, and Yingqian Hu. 2026. Efficient influential community search over dynamic graphs. *Proceedings of the ACM on Management of Data* 4, 1 (2026), 1–28.

[34] Maxim Sviridenko. 2004. A note on maximizing a submodular set function subject to a knapsack constraint. *Operations Research Letters* 32, 1 (2004), 41–43.

[35] Jie Tang, Jing Zhang, Limin Yao, Juanzi Li, Li Zhang, and Zhong Su. 2008. Arnet-Miner: Extraction and mining of academic social networks. In *Proceedings of the 14th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*. ACM, 990–998.

[36] Youze Tang, Yanchen Shi, and Xiaokui Xiao. 2015. Influence maximization in near-linear time: A martingale approach. In *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data (SIGMOD)*. ACM, New York, NY, USA, 1539–1554.

[37] Youze Tang, Xiaokui Xiao, and Yanchen Shi. 2014. Influence maximization: Near-optimal time complexity meets practical efficiency. In *Proceedings of the 2014 ACM SIGMOD International Conference on Management of Data (SIGMOD)*. ACM, New York, NY, USA, 75–86.

[38] Long Teng, Yanhao Wang, Zhe Lin, and Fei Yu. 2025. Topic-aware most influential community search in social networks. *Neurocomputing* 638 (2025), 130173.

[39] Yanping Wu, Jun Zhao, Renjie Sun, Chen Chen, and Xiaoyang Wang. 2021. Efficient personalized influential community search in large networks. *Data Science and Engineering* 6, 3 (2021), 310–322.

[40] Carl Yang, Yuxin Xiao, Yu Zhang, Yizhou Sun, and Jiawei Han. 2020. Heterogeneous network representation learning: A unified framework with survey and benchmark. *IEEE Transactions on Knowledge and Data Engineering* (2020).

[41] Kai Yao and Lijun Chang. 2021. Efficient size-bounded community search over large networks. *Proceedings of the VLDB Endowment* 14, 8 (2021), 1441–1453.

[42] Xiao Yu, Xiang Ren, Yizhou Sun, Bradley Sturt, Urvashi Khandelwal, Quanquan Gu, Brandon Norick, and Jiawei Han. 2013. Recommendation in heterogeneous information networks with implicit user feedback. In *Proceedings of the 7th ACM Conference on Recommender Systems (RecSys)*. ACM, New York, NY, USA, 347–350.

[43] Fan Zhang, Haicheng Guo, Dian Ouyang, Shiyu Yang, Xuemin Lin, and Zhihong Tian. 2024. Size-constrained community search on large networks: An effective and efficient solution. *IEEE Transactions on Knowledge and Data Engineering* 36, 1 (2024), 356–371.

[44] Nan Zhang, Yutong Ye, Xiang Lian, and Mingsong Chen. 2024. Top-L most influential community detection over social networks. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE, IEEE, 5767–5779.

[45] Qifan Zhang and Shengxin Liu. 2023. Efficient exact minimum k-core search in real-world graphs. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management (CIKM)*. ACM, New York, NY, USA.

[46] Wentong Zhang, Kaiyu Feng, Lanting Fang, Junghoon Kim, Kaibo Zhang, Dahee Kim, Shuliang Wang, Ye Yuan, and Guoren Wang. 2025. Budgeted strong community search in heterogeneous graphs. *Proceedings of the ACM on Management of Data* 3, 6, Article 303 (2025), 27 pages.

[47] Yingli Zhou, Yixiang Fang, Wensheng Luo, and Yunming Ye. 2023. Influential community search over large heterogeneous information networks. *Proceedings of the VLDB Endowment* 16, 8 (2023), 2047–2060.